Bases de Datos · Proyecto final

Reseñas de Amazon con MySQL, MongoDB y Neo4j

Diseño, carga, visualización y análisis en grafo de unas 770.000 reseñas de productos

Autores Juan Tabuyo Castell
Jaime Tuesta de Simón

Titulación Grado en Ingeniería Matemática e Inteligencia Artificial (2.º curso)

Centro ICAI, Universidad Pontificia Comillas

Fecha Abril de 2026

Índice

1 Introducción **2**

2 Diseño y carga de datos **3**

2.1 Esquema relacional (MySQL) . . . 3

2.2 Justificación del diseño . . . 3

2.3 Reparto entre MySQL y MongoDB . . . 4

2.4 Carga de datos e índices . . . 4

3 Aplicación de acceso y visualización **5**

4 Análisis en grafo con Neo4j **7**

4.1 Similitud entre usuarios . . . 7

4.2 Artículos aleatorios y sus usuarios . . . 7

4.3 Usuarios de varias categorías . . . 7

4.4 Artículos populares y artículos en común . . . 8

5 Inserción de nuevos datos **8**

6 Visualización con Power BI y recomendación **9**

6.1 Gráficas en Power BI . . . 9

6.2 Modelo de recomendación propuesto . . . 10

6.3 Recomendación de artículos populares no consumidos . . . 10

1 Introducción

El proyecto construye una plataforma de gestión y análisis de reseñas de Amazon que combina tres sistemas de bases de datos: MySQL para los datos estructurados, MongoDB para los textos y Neo4j para el análisis en forma de grafo. Se trabaja con los subconjuntos *5-core* (cada usuario y cada producto tienen al menos cinco reseñas) de cuatro categorías: juguetes (167.597 reseñas), videojuegos (231.780), música digital (64.706) e instrumentos musicales (10.261). Más adelante se añade una quinta categoría, deportes y aire libre (296.337 reseñas).

Cada línea de los ficheros es un objeto JSON con una reseña: el identificador y el nombre del autor (`reviewerID`, `reviewerName`), el identificador del producto (`asin`), la nota (`overall`), los votos de utilidad (`helpful`), el texto y el resumen (`reviewText`, `summary`) y la fecha en dos formatos (`unixReviewTime`, `reviewTime`).

Todo el proyecto está escrito en Python, en seis módulos:

Módulo	**Función**
`configuracion.py`	Credenciales, nombres de las bases de datos y rutas de los ficheros.
`load_data.py`	Creación de las bases de datos, tablas, colección e índices, y carga de los cuatro ficheros.
`menu_visualizacion.py`	Menú de visualización sobre MySQL y MongoDB.
`neo4JProyecto.py`	Menú de análisis en grafo con Neo4j.
`inserta_dataset.py`	Inserción de un nuevo dataset en la base de datos existente.
`recomendacion_populares.py`	Recomendación de los artículos más populares no consumidos.

2 Diseño y carga de datos

2.1 Esquema relacional (MySQL)

La base de datos `ProyectoBBDD` de MySQL tiene tres tablas (claves primarias subrayadas):

- USUARIOS(reviewerID, reviewerName)
- ARTICULOS(asin, categoria)
- REVIEWS(review_id, reviewerID, asin, overall, helpful_1, helpful_2, unixReviewTime, reviewTime, categoria)
  con `reviewerID` clave foránea hacia USUARIOS y `asin` clave foránea hacia ARTICULOS.

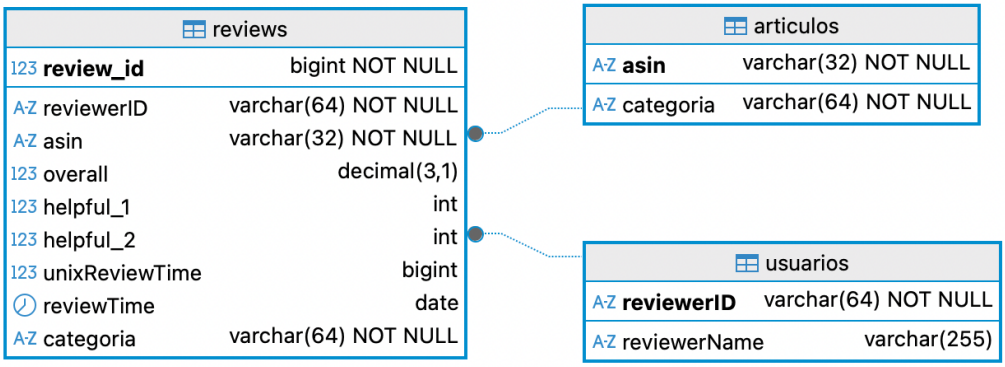


Figura 1. Esquema relacional de la base de datos MySQL.

2.2 Justificación del diseño

Separación en tres tablas. Cada reseña del JSON original mezcla información de tres entidades distintas: el usuario que la escribe (`reviewerID`, `reviewerName`), el artículo reseñado (`asin`, categoría) y la propia reseña (nota, utilidad, fechas). Como un mismo usuario puede escribir reseñas en varias categorías y un mismo artículo puede recibir muchas reseñas, USUARIOS y ARTICULOS se extraen a tablas propias para evitar redundancia y cumplir la tercera forma normal. REVIEWS queda como tabla de hechos con dos claves foráneas hacia ambas entidades.

Clave primaria de REVIEWS. No hay garantía de que la pareja usuario–artículo, ni siquiera junto con la fecha, sea única en los datos. Por eso se añade un identificador autoincremental `review_id` como clave primaria artificial, que además es más eficiente en los JOIN que una clave compuesta.

Tipos de datos. `reviewTime` se convierte del texto «MM DD, YYYY» al tipo `DATE`, como exige el enunciado. `unixReviewTime` se mantiene como `BIGINT` porque permite ordenar cronológicamente sin conversiones. La nota se guarda como `DECIMAL(3,1)` y el campo `helpful` se desdobla en dos enteros (`helpful_1`, votos útiles, y `helpful_2`, votos totales) para poder consultarlos por separado.

Categoría en REVIEWS. La categoría está en ARTICULOS, pero se repite en REVIEWS para filtrar por categoría en las visualizaciones sin necesidad de un JOIN. Es una desnormalización controlada, justificada por el patrón de consultas esperado.

2.3 Reparto entre MySQL y MongoDB

En **MySQL** se almacena toda la información estructurada y numérica: identificadores (`reviewerID`, `asin`, `review_id`), nombre del usuario, categoría, nota, utilidad y las dos fechas. Son datos de tamaño fijo, con relaciones claras entre entidades, sobre los que se hacen consultas relacionales (JOIN, GROUP BY, agregaciones por año, categoría o usuario) para las gráficas del menú de visualización.

En **MongoDB** (colección `reviews_texto`) se almacenan los campos de texto libre, `reviewText` y `summary`, junto con `reviewerID`, `asin`, `unixReviewTime` y la categoría como metadatos para poder filtrar y relacionar cada documento con su reseña. Son campos de longitud muy variable y contenido no estructurado, que encajan mejor en una base de datos documental. En el proyecto se usan para la nube de palabras por categoría, que se resuelve con una consulta `find` que proyecta `summary`, sin necesidad de JOIN.

De este modo, todos los campos del JSON original quedan almacenados: MySQL concentra los datos sobre los que se hacen consultas relacionales, y MongoDB, los campos textuales.

2.4 Carga de datos e índices

La creación de la base de datos, las tablas, la colección y los índices, y la inserción de los datos, se realizan desde Python ejecutando `load_data.py`. Las credenciales, los nombres de las bases de datos y las rutas de los ficheros se definen en `configuracion.py`, de modo que basta con modificar ese fichero para cargar los datos en otro equipo.

Los ficheros se leen **línea a línea** con un generador, sin cargar nunca el fichero completo en memoria, y las inserciones se agrupan en lotes de 5.000 reseñas (`executemany` en MySQL e `insert_many` en MongoDB) para reducir el número de viajes a cada base de datos. Como un mismo usuario puede tener reseñas en varias categorías, antes de cargar cada fichero se leen los usuarios y artículos ya existentes y se mantienen en conjuntos, de forma que cada usuario y cada artículo se insertan una sola vez.

Al terminar la carga se crean índices para acelerar las consultas de visualización:

- MySQL (tabla REVIEWS): `categoria`, `asin`, `reviewerID`, `overall` y `unixReviewTime`.
- MongoDB (colección `reviews_texto`): `categoria`, `reviewerID` y `asin`.

3 Aplicación de acceso y visualización

La aplicación `menu_visualizacion.py` permite consultar y visualizar distintos aspectos de los datos mediante un menú por terminal que se repite hasta que el usuario elige salir (Figura 2). Las categorías disponibles se leen de la propia base de datos al arrancar, de modo que la aplicación incluye automáticamente las que se añadan después.

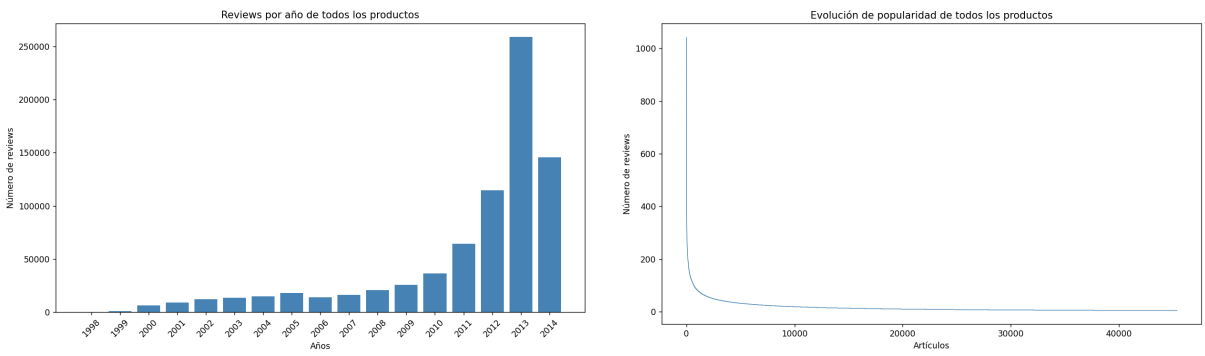
```
======================================================
  MENÚ DE VISUALIZACIÓN - PROYECTO BBDD
======================================================
  1. Evolución de reviews por años
  2. Evolución de la popularidad de los artículos
  3. Histograma por nota
  4. Evolución de reviews a lo largo del tiempo
  5. Histograma de reviews por usuario
  6. Nube de palabras (por categoría)
  7. Top 15 usuarios con más reviews
  8. Salir
======================================================
Selecciona una opción:
```

Figura 2. Menú de visualización.

El menú ofrece siete gráficas:

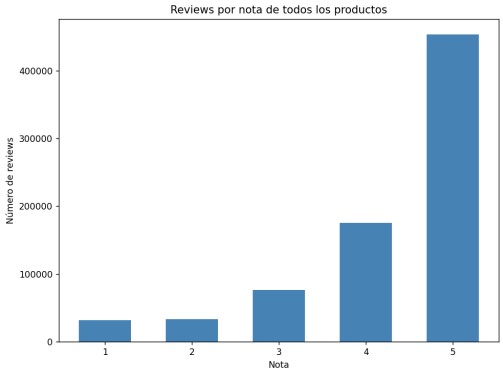
1. **Reseñas por año**, por categoría o para todas.
2. **Evolución de la popularidad**: artículos ordenados de mayor a menor número de reseñas.
3. **Histograma por nota**, por categoría, para todas o para un artículo concreto. Si el artículo introducido no existe, se informa al usuario.
4. **Evolución de las reseñas a lo largo del tiempo**: número acumulado de reseñas de cada categoría.
5. **Histograma de reseñas por usuario**: cuántos usuarios han escrito cada número de reseñas.
6. **Nube de palabras por categoría**, a partir del campo `summary` de MongoDB, considerando solo las palabras de más de tres letras y generada con la librería `wordcloud`.
7. **Los 15 usuarios con más reseñas**, como visualización adicional.

Las consultas numéricas se realizan sobre MySQL, que almacena los campos necesarios para filtros, agrupaciones y ordenaciones, y la nube de palabras, sobre MongoDB. Las Figuras 3–5 muestran las gráficas obtenidas con todos los productos.

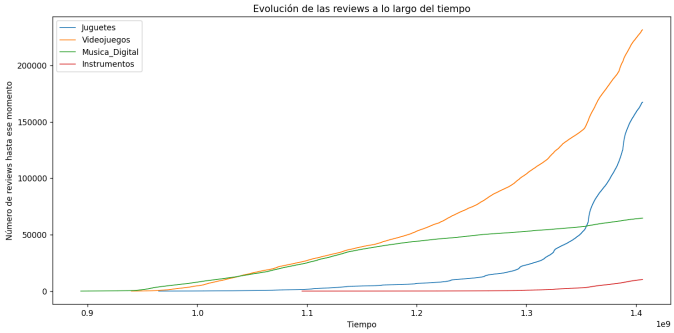


(a) Reseñas por año. **(b)** Popularidad de los artículos.

Figura 3. Reseñas por año y evolución de la popularidad de los artículos.

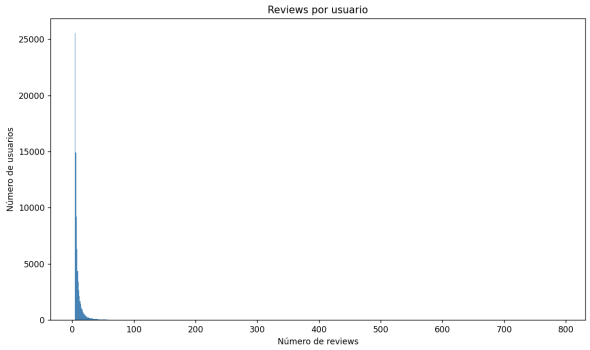


(a) Reseñas por nota.

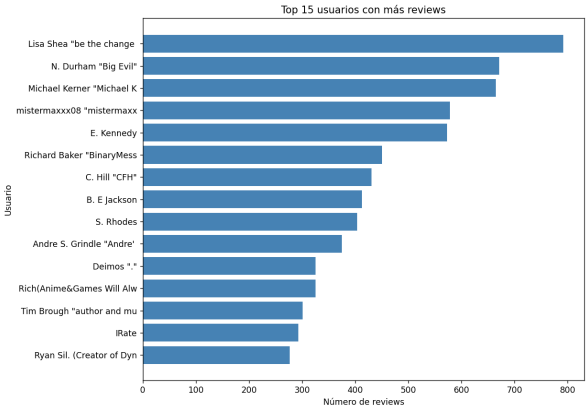


(b) Evolución de las reseñas a lo largo del tiempo.

Figura 4. Histograma por nota y evolución acumulada de las reseñas por categoría.



(a) Reseñas por usuario.



(b) Los 15 usuarios con más reseñas.



(c) Nube de palabras de la categoría de instrumentos.

Figura 5. Reseñas por usuario, usuarios más activos y nube de palabras.

4 Análisis en grafo con Neo4j

La aplicación `neo4JProyecto.py` es otro menú por terminal (Figura 6) que obtiene datos de MySQL y los carga en Neo4j con distintas representaciones en forma de grafo. Cada opción limpia antes la base de datos de Neo4j para no mezclar resultados y, al terminar la carga, avisa al usuario para que pueda consultarla en Neo4j Browser.

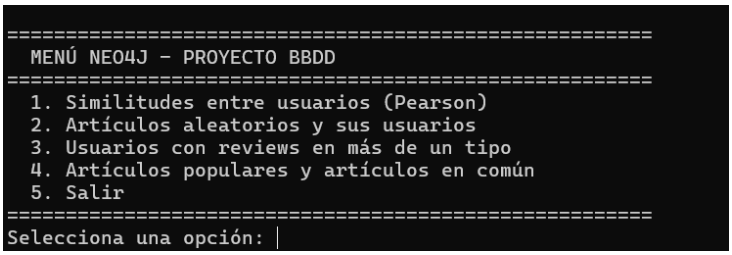


Figura 6. Menú de Neo4j.

4.1 Similitud entre usuarios

Se obtienen los 30 usuarios con más reseñas (el número se define en `configuracion.py`) y se calcula la similitud entre cada par mediante la correlación de Pearson sobre los artículos que ambos han puntuado:

$$PC(u,v) = \frac{\sum_{i \in I_{uv}} (r_{ui} - \bar{r}_u)(r_{vi} - \bar{r}_v)}{\sqrt{\sum_{i \in I_{uv}} (r_{ui} - \bar{r}_u)^2} \sqrt{\sum_{i \in I_{uv}} (r_{vi} - \bar{r}_v)^2}},$$

donde I_{uv} es el conjunto de artículos puntuados por ambos usuarios y $\bar{r}_u$, $\bar{r}_v$ son las medias de todas las valoraciones de cada usuario. Solo se guardan las similitudes de los pares con algún artículo en común. En Neo4j, cada usuario es un nodo y cada similitud, una relación con su valor; como la similitud es simétrica, se crea una relación por par y se consulta sin dirección. Una consulta Cypher muestra por pantalla el usuario con más vecinos.

4.2 Artículos aleatorios y sus usuarios

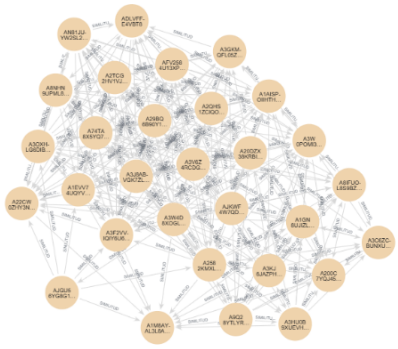
El usuario elige una categoría y un número n de artículos, que se seleccionan al azar. Se cargan en Neo4j esos artículos y todos los usuarios que los han puntuado, unidos por relaciones `HA_PUNTUADO` con la nota y el momento de la reseña como propiedades.

4.3 Usuarios de varias categorías

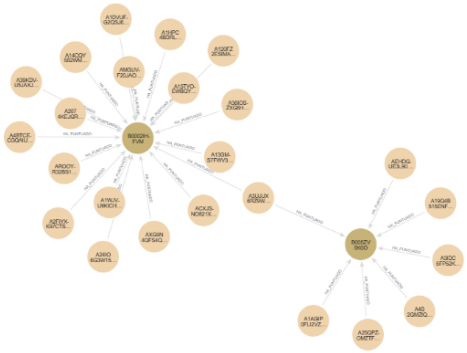
Se toman los 400 primeros usuarios ordenados por nombre y se seleccionan los que han puntuado artículos de al menos dos categorías distintas. En Neo4j se representan como nodos de usuario enlazados con nodos de tipo de artículo, y cada enlace indica cuántos artículos de ese tipo ha puntuado el usuario.

4.4 Artículos populares y artículos en común

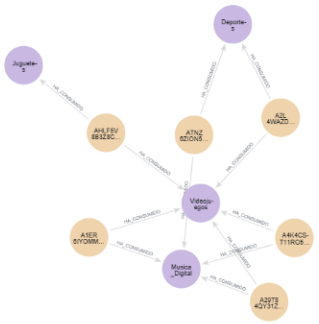
Se seleccionan los 5 artículos más populares de entre los que tienen menos de 40 reseñas y se cargan con todos los usuarios que los han puntuado. Además, se crean enlaces entre cada par de usuarios con artículos en común, con el número de artículos compartidos como propiedad.



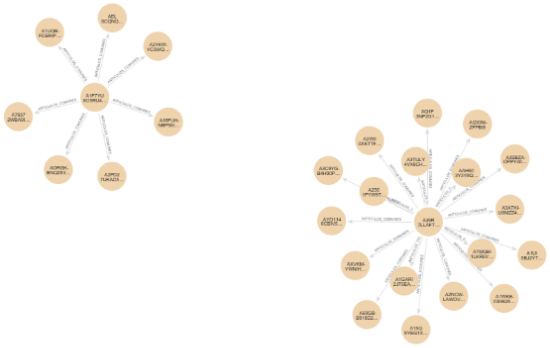
(a) Similitud entre los 30 usuarios con más reseñas.



(b) Artículos aleatorios y sus usuarios.



(c) Usuarios con reseñas en varias categorías.



(d) Artículos populares y usuarios con artículos en común.

Figura 7. Grafos obtenidos en Neo4j Browser con cada opción del menú.

5 Inserción de nuevos datos

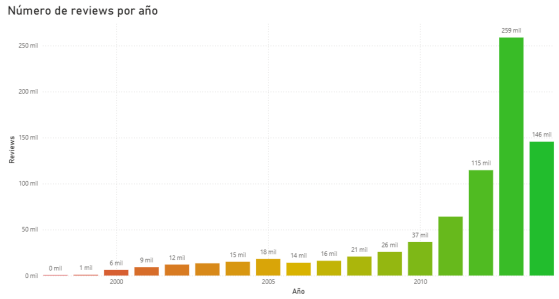
El script `inserta_dataset.py` añade un nuevo dataset a la base de datos ya creada, sin eliminar ni modificar los datos existentes. Se ha elegido el fichero de deportes y aire libre (*Sports and Outdoors*), que se incorpora como categoría «Deportes».

El proceso comienza con dos validaciones: que el fichero exista en la ruta configurada y que la categoría no esté ya en la base de datos, para evitar cargar dos veces el mismo dataset. A continuación se leen los identificadores de los usuarios y artículos existentes. Como un mismo usuario puede haber escrito reseñas en distintas categorías, los usuarios y artículos que ya estaban se reutilizan y solo se insertan los nuevos; todas las reseñas del fichero se insertan en MySQL y MongoDB con la misma estructura que en la carga inicial. Al terminar, el programa muestra cuántas reseñas, usuarios nuevos, usuarios reutilizados y artículos nuevos se han insertado, y el estado global de la base de datos.

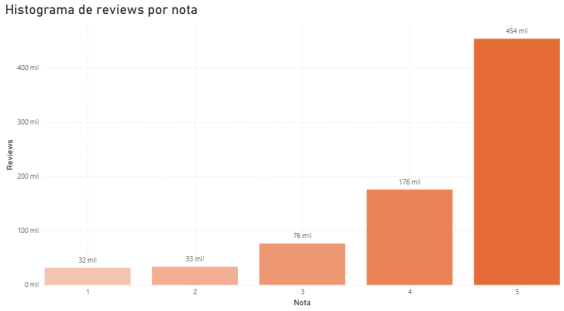
6 Visualización con Power BI y recomendación

6.1 Gráficas en Power BI

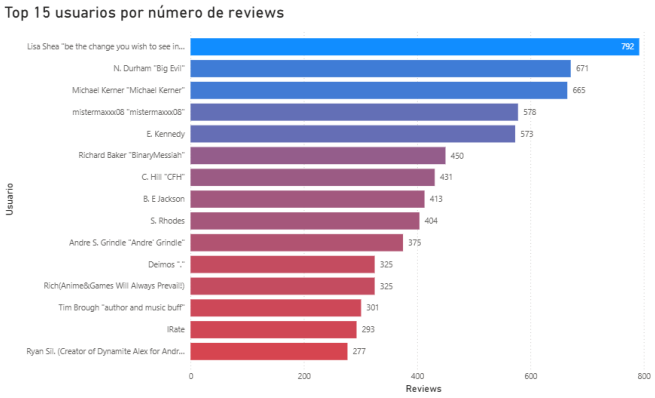
Tres de las gráficas obtenidas con Python (reseñas por año, histograma por nota y los 15 usuarios con más reseñas) se han reproducido con Power BI (Figura 8).



(a) Reseñas por año.



(b) Reseñas por nota.



(c) Los 15 usuarios con más reseñas.

Figura 8. Gráficas reproducidas con Power BI.

6.2 Modelo de recomendación propuesto

Se propone un sistema de recomendación que sugiera a cada usuario artículos que todavía no ha consumido, utilizando principalmente los datos estructurados de MySQL (`reviewerID`, `asin`, `overall`, `categoria`). A partir de estas interacciones se construiría una matriz usuario–artículo, cuyos valores serían las notas (vacíos si no hay interacción), y se aplicaría un modelo de filtrado colaborativo, que permite predecir qué productos pueden interesar a un usuario a partir de sus valoraciones previas y de patrones de comportamiento similares.

Con las predicciones, se eliminarían los artículos ya consumidos por el usuario y se generaría un ranking con las recomendaciones más relevantes, con la opción de filtrar por categoría. Como mecanismo de respaldo para usuarios con poco historial, se recurriría a la popularidad por categoría, devolviendo los artículos más populares que el usuario aún no haya consumido. La Figura 9 resume el proceso.

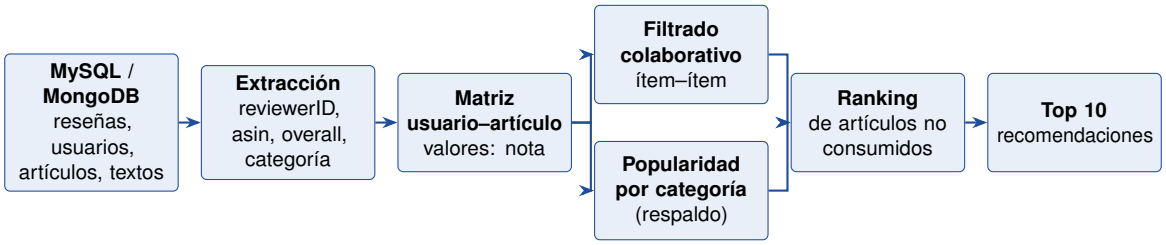


Figura 9. Diagrama del modelo de recomendación propuesto.

6.3 Recomendación de artículos populares no consumidos

El script `recomendacion_populares.py` implementa el mecanismo de respaldo: dado el identificador original de un usuario y una categoría, devuelve los 10 artículos más populares de esa categoría que el usuario todavía no ha consumido. Se resuelve con una única consulta SQL sobre `REVIEWS` que filtra por categoría, excluye con `NOT EXISTS` los artículos que el usuario ya ha reseñado, agrupa por artículo y ordena por número de reseñas (con la nota media y el `asin` como criterios de desempate, para que el resultado sea estable). El programa comprueba antes que el usuario exista y ofrece las categorías disponibles en la base de datos.

```
======================================================================
RECOMENDACIONES
======================================================================
Usuario   : A00046902LP5YSDV0VVNF
Categoría : Videojuegos

Top artículos más populares no consumidos:
  1. ASIN: B00BGA9WK2 | reviews: 802 | media overall: 4.38
  2. ASIN: B0015AARJI | reviews: 652 | media overall: 4.60
  3. ASIN: B007CM0K86 | reviews: 517 | media overall: 4.60
  4. ASIN: B002VBWIP6 | reviews: 465 | media overall: 4.75
  5. ASIN: B0009VXBAQ | reviews: 447 | media overall: 4.46
  6. ASIN: B0050SYX8W | reviews: 441 | media overall: 4.31
  7. ASIN: B000B9RI14 | reviews: 425 | media overall: 4.82
  8. ASIN: B00178630A | reviews: 379 | media overall: 2.65
  9. ASIN: B007XVTR3K | reviews: 365 | media overall: 3.61
 10. ASIN: B00005NZ1G | reviews: 356 | media overall: 4.53
```

Figura 10. Diez recomendaciones de la categoría de videojuegos para un usuario.